

特征哈希

特征哈希 (Feature hashing, 也称 hashing trick) 通过哈希函数直接确定特征在样本矩阵中的列索引。scikit-learn 的 `sklearn.feature_extraction.FeatureHasher` 使用这一方法, 将输入特征转换为向量, 速度快、内存占用低。它不需要像其他向量化器那样, 为训练时遇到的特征建立并保存哈希表。

这种做法的代价是难以追溯输出列对应的原始特征: `FeatureHasher` 不保存输入特征的原貌, 也没有 `inverse_transform` 方法。

输入与输出

`FeatureHasher` 接受哪种输入, 由构造参数 `input_type` 决定。输入可以是映射对象, 例如 Python 的 `dict` 及 `collections` 模块中的字典变体, 也可以是 `(feature, value)` 对或字符串。

映射对象会按一组 `(feature, value)` 对处理。字符串特征的隐含值为 1, 因此 `['feat1', 'feat2', 'feat3']` 等价于 `[('feat1', 1), ('feat2', 1), ('feat3', 1)]`。同一特征在一个样本中多次出现时, 其数值会相加, 例如 `('feat', 2)` 和 `('feat', 3.5)` 合并后为 `('feat', 5.5)`。

输出始终是 CSR 格式的 `scipy.sparse` 稀疏矩阵。

哈希冲突与符号处理

不同特征可能被映射到同一列, 即发生哈希冲突。为减轻冲突带来的误差, `FeatureHasher` 使用带符号的哈希函数, 由哈希值的符号决定特征写入输出矩阵时的符号。这样, 冲突造成的误差更可能相互抵消, 而不是不断累积; 每个输出特征值的期望均值为零。

这一机制默认通过 `alternate_sign=True` 启用, 在哈希空间较小 (`n_features < 10000`) 时尤其有用。哈希空间较大时, 可以设置 `alternate_sign=False` 关闭符号交替机制, 以便将输出用于要求非负输入的估计器或特征选择方法, 例如 `MultinomialNB` 和 `chi2`。这类下游方法要求输入值非负, 因此传给哈希器的特征值本身也应满足这一条件。

文本任务中的用法

特征哈希可用于文档分类。不过, 与 `CountVectorizer` 不同, `FeatureHasher` 除了将 Unicode 编码为 UTF-8, 不会分词, 也不会执行其他预处理。需要将分词和哈希转换结合起来时, 可使用 `HashingVectorizer`。

例如, 在词级自然语言处理任务中, 可以根据 `(token, part_of_speech)` 对提取特征, 其中两项分别表示词元和词性。下面的 Python 生成器函数提取数字标记、词元、词元与词性的组合、大小写标记和词性:

```
def token_features(token, part_of_speech):
    if token.isdigit():
        yield "numeric"
    else:
        yield "token={}".format(token.lower())
        yield "token,pos={},{}".format(token, part_of_speech)
    if token[0].isupper():
        yield "uppercase_initial"
    if token.isupper():
        yield "all_uppercase"
    yield "pos={}".format(part_of_speech)
```

给定词元序列 `corpus` 和词性标注函数 `pos_tagger`, 可以构造 `raw_X`, 再调用 `FeatureHasher.transform`:

```
from sklearn.feature_extraction import FeatureHasher

raw_X = (token_features(tok, pos_tagger(tok)) for tok in corpus)

hasher = FeatureHasher(input_type='string')
X = hasher.transform(raw_X)
```

得到的 `X` 是 `scipy.sparse` 稀疏矩阵。这里使用生成器表达式按需提取特征: 只有哈希器需要数据时, 才会处理相应词元。

实现与规模限制

`FeatureHasher` 使用 `MurmurHash3` 的有符号 32 位变体 (signed 32-bit variant)。受这一实现及 `scipy.sparse` 的限制, 支持的最大特征数为 $2^{31} - 1$ 。

Weinberger 等人在 2009 年提出的 hashing trick 原始形式使用两个独立的哈希函数 h 和 ξ ，分别确定特征的列索引和符号。FeatureHasher 的实现则假设 MurmurHash3 的符号位与其他位相互独立。

哈希值通过简单的取模运算转换为列索引，因此 `n_features` 宜设为 2 的幂；否则，特征无法均匀映射到各列。

参考资料

- Kilian Weinberger、Anirban Dasgupta、John Langford、Alex Smola 和 Josh Attenberg (2009)。Feature hashing for large scale multitask learning。ICML 会议论文集。
- MurmurHash3 实现。